

TEAM HOLANDA

Titanic project

TA: Esraa Ali

Name	ID
احمد سامح حسن احمد محمد شوشه	20241700031
انس سامي محمد سلميان	20241700147
مهند خالد عبدالحليم عمران	20241701073
احمد عمرو احمد عبدالحليم	20241700050
احمد طارق محمد عبدالعليم	20241700040
ادهم هيثم نبيل عبدالعزيز	20241700098
احمد شريف منصور السعيد	20241700036

1. Project Overview

This project builds a complete machine learning pipeline to predict whether a Titanic passenger survived, based on features such as age, sex, passenger class, fare, and more. The pipeline covers data loading, cleaning, exploratory data analysis, feature engineering, encoding, scaling, training of five classifiers, hyperparameter tuning, and final model selection.

Dataset	Rows	Columns	Target
Train	891	12	Survived (0/1)
Test	418	11	— (no label)

2. Imports & Libraries

The notebook begins by importing all required libraries:

Library	Purpose
<code>pandas (pd)</code>	Data loading, cleaning, and manipulation (DataFrames)
<code>numpy (np)</code>	Numerical operations — log transforms, array math
<code>matplotlib.pyplot (plt)</code>	Low-level plotting — axes, figures, subplots
<code>seaborn (sns)</code>	High-level statistical plots — countplot, heatmap, histplot
<code>sklearn.model_selection</code>	<code>train_test_split</code> (split data), <code>GridSearchCV</code> (tuning), <code>learning_curve</code>
<code>sklearn.preprocessing</code>	<code>StandardScaler</code> (scale features), <code>OrdinalEncoder</code> (ordinal encoding)
<code>sklearn.linear_model</code>	<code>LogisticRegression</code> — baseline linear classifier
<code>sklearn.tree</code>	<code>DecisionTreeClassifier</code> — tree-based rule learner
<code>sklearn.svm</code>	<code>SVC</code> — Support Vector Classifier
<code>sklearn.ensemble</code>	<code>RandomForestClassifier</code> — ensemble of trees
<code>sklearn.neighbors</code>	<code>KNeighborsClassifier</code> — distance-based classifier
<code>sklearn.metrics</code>	<code>accuracy_score</code> , <code>confusion_matrix</code> , <code>classification_report</code>

3. Data Loading

The Titanic training and test CSV files are loaded using `pd.read_csv()`. The training set contains 891 rows and 12 columns including the target `Survived` column. The test set has 418 rows and 11 columns (no `Survived` label). `train.info()` reveals data types and null counts; `train.describe()` provides summary statistics for numeric columns.

4. Data Cleaning

Before modelling, missing values must be handled. The table below summarises which columns contained nulls and how they were treated:

Column	Missing in Train	Strategy	Reason
Age	~177	Median imputation	Robust to outliers
Fare (test)	1	Median imputation	Single missing value
Embarked	2	Mode imputation	Categorical — most frequent
Cabin	~687 (77%)	Dropped	Too sparse to impute usefully

5. Exploratory Data Analysis (EDA)

Before building models, we explore the data visually to understand distributions, class balance, and relationships between features and the target variable.

5.1 Overall Survival Count

The bar chart below shows the class distribution. Approximately 62% of passengers did not survive, revealing a moderate class imbalance in the dataset.

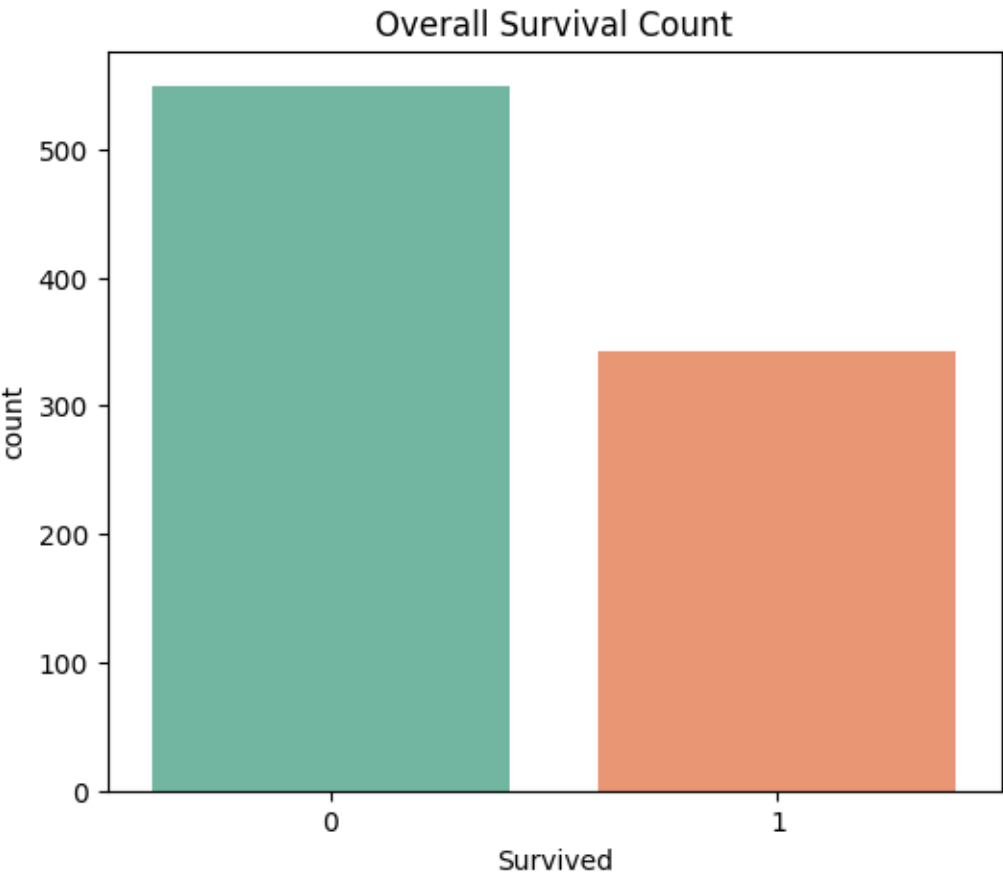


Figure 1: Overall count of survivors vs non-survivors

5.2 Survival by Categorical Features

The three panels below show survival split by passenger class (Pclass), sex, and port of embarkation (Embarked). Female passengers and first-class passengers show dramatically higher survival rates, consistent with the "women and children first" protocol.

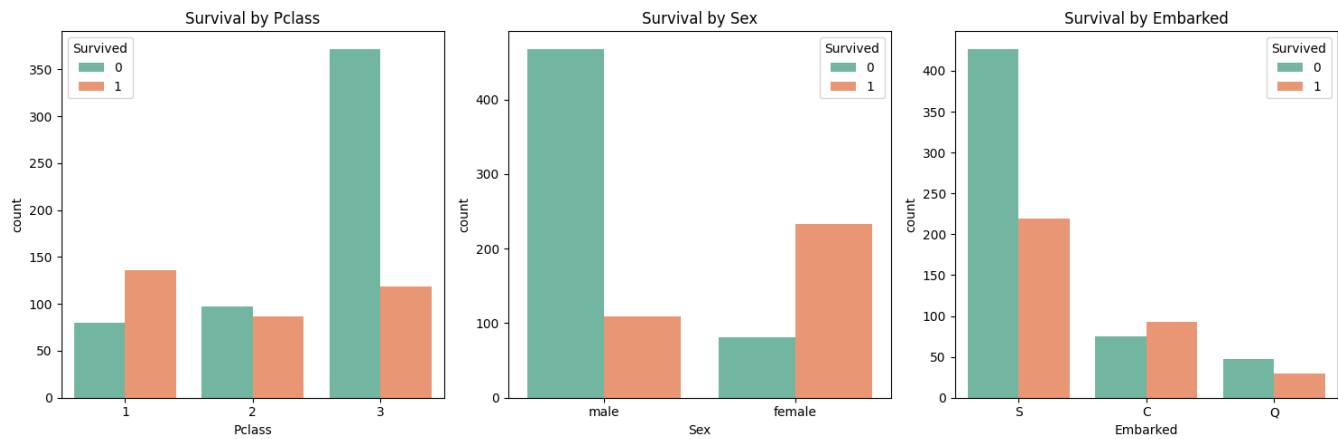


Figure 2: Survival counts by Pclass, Sex, and Embarked

5.3 Age and Fare Distributions

Histograms with KDE curves reveal that Age follows an approximately normal distribution centred around ~30 years, while Fare is heavily right-skewed with a long tail from expensive first-class cabins — motivating a log transformation later.

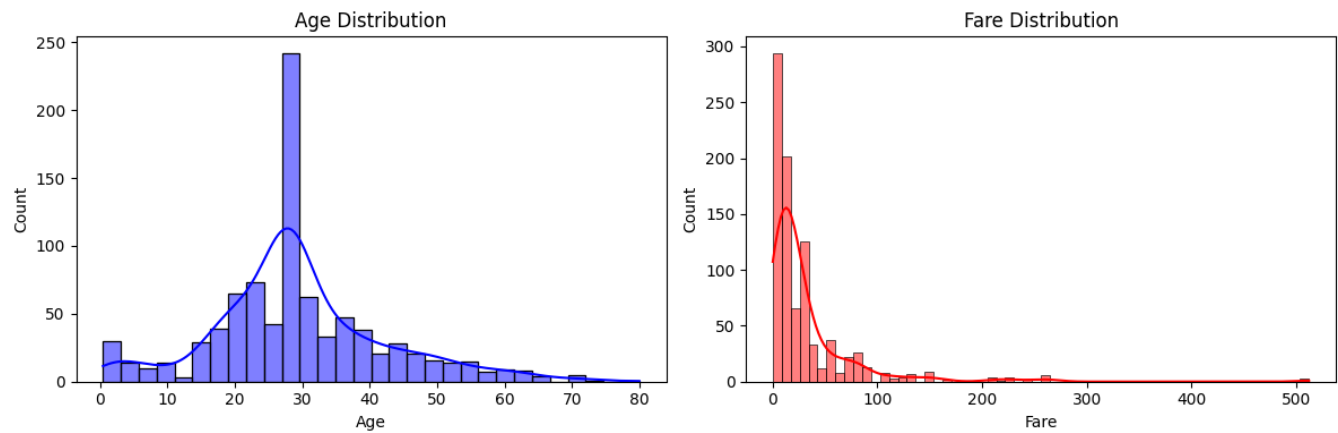


Figure 3: Age and Fare distributions with KDE curves

5.4 Correlation Heatmap

The Pearson correlation heatmap between all numeric features reveals key relationships. Notably, Pclass shows a moderate negative correlation with Survived (higher class number = lower survival), while Fare shows a positive correlation.

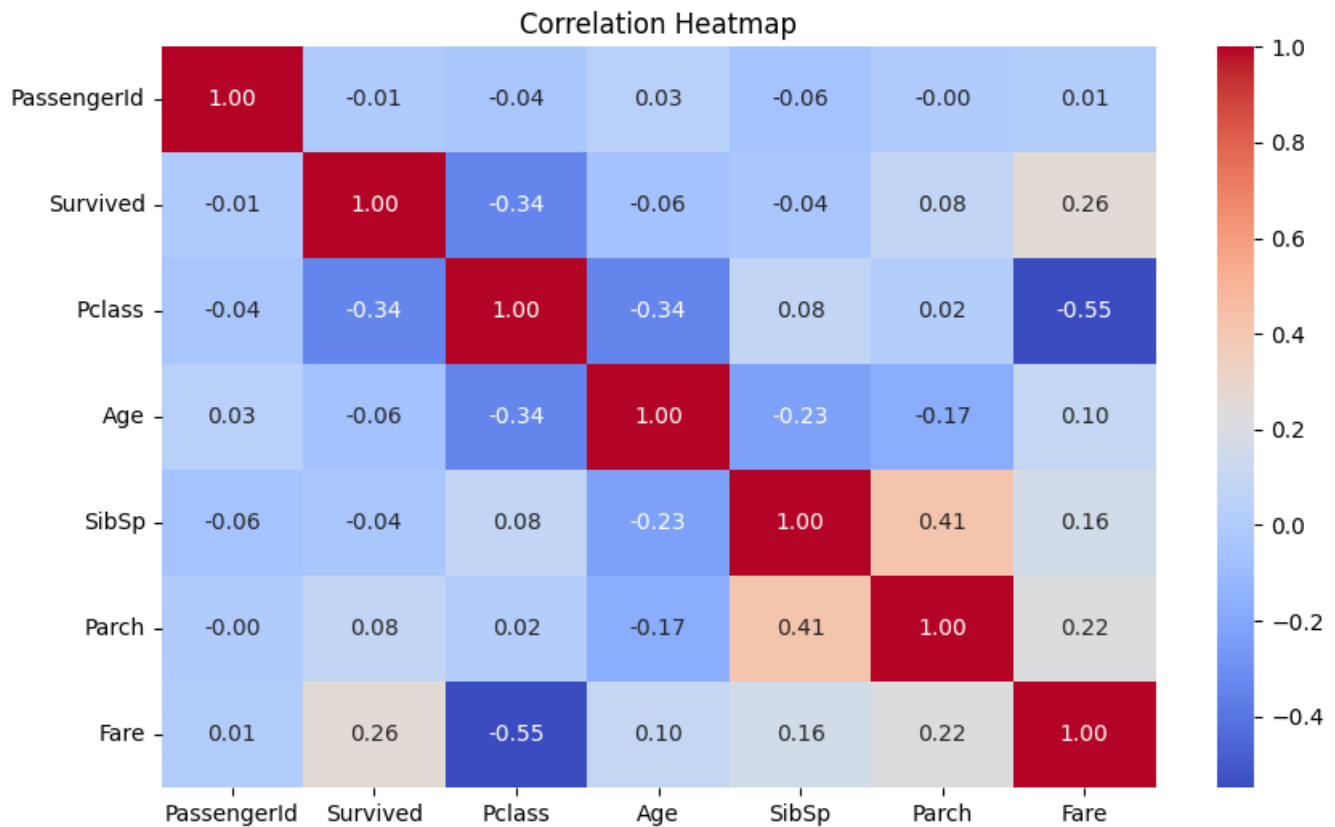


Figure 4: Pearson correlation heatmap of numeric features

6. Feature Engineering

Raw features are transformed and new features are created to improve model performance.

6.1 Title Extraction from Name

Using the regular expression `r',\s*([^\.]+)\.'`, titles such as Mr, Mrs, Miss, Master, Dr, Rev are extracted from passenger names. The value counts of extracted titles are shown below:

Title	Count
Mr	517
Miss	182
Mrs	125
Dr	7
Rev	6
Major	2
Ms	2
Lady	1
Col	1
Mlle	1

Mme	1
Don	1
Sir	1
Capt	1
The countess	1
Jonkheer	1

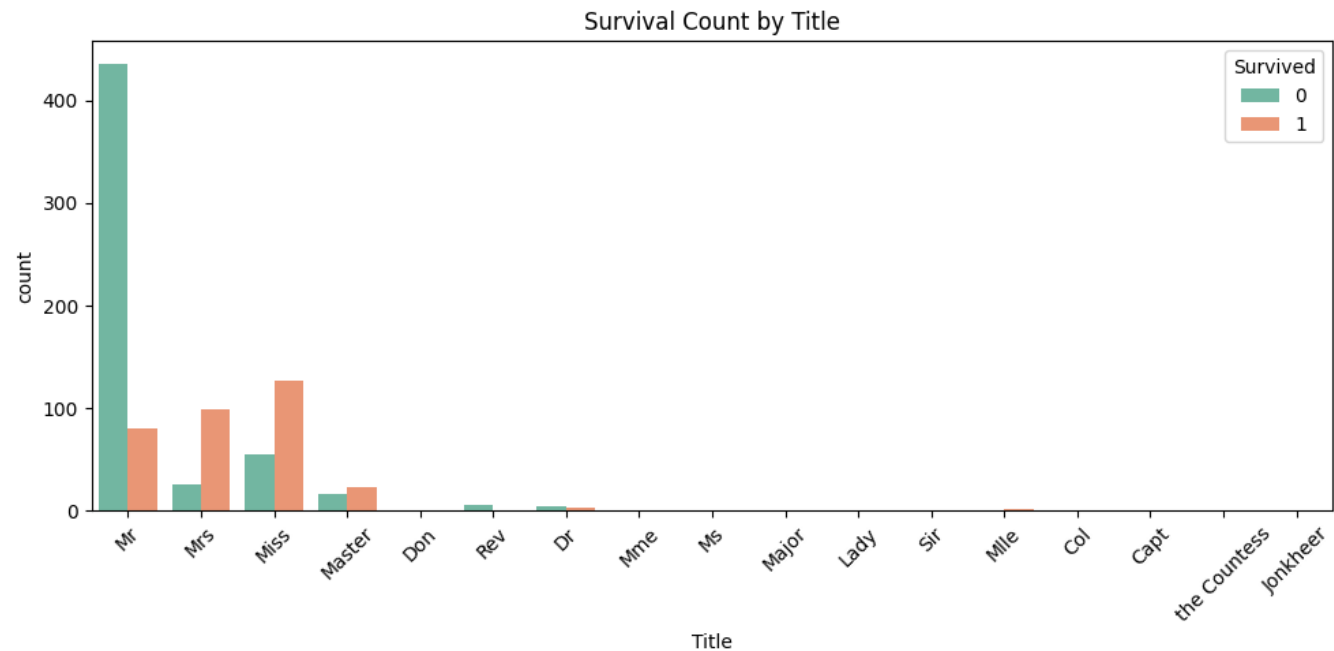


Figure 5: Survival count by title

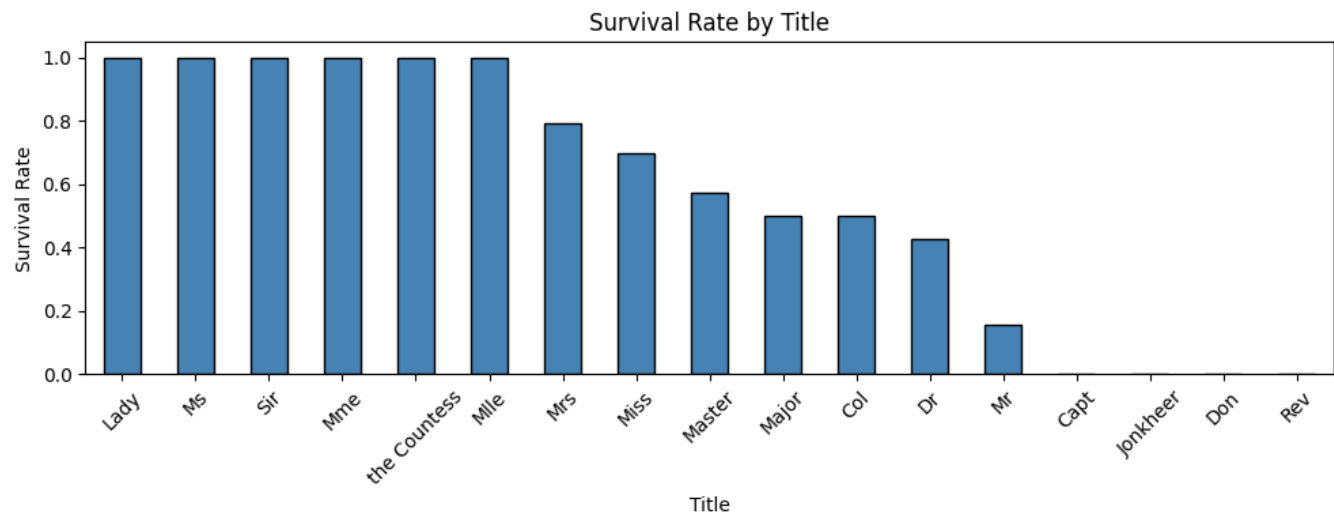


Figure 6: Survival rate by title

Rare titles (Dr, Rev, Col, etc.) are consolidated into a single "Rare" category because they have too few examples for the model to learn reliable patterns. Only Mr, Miss, Mrs, and Master are kept as-is.

6.2 Family Size & IsAlone

FamilySize = SibSp + Parch + 1 captures the total number of family members aboard including the passenger themselves. IsAlone = 1 when FamilySize equals 1, indicating a solo traveller. Research shows solo travellers had different survival odds than family groups.

6.3 Age Distribution by Survival

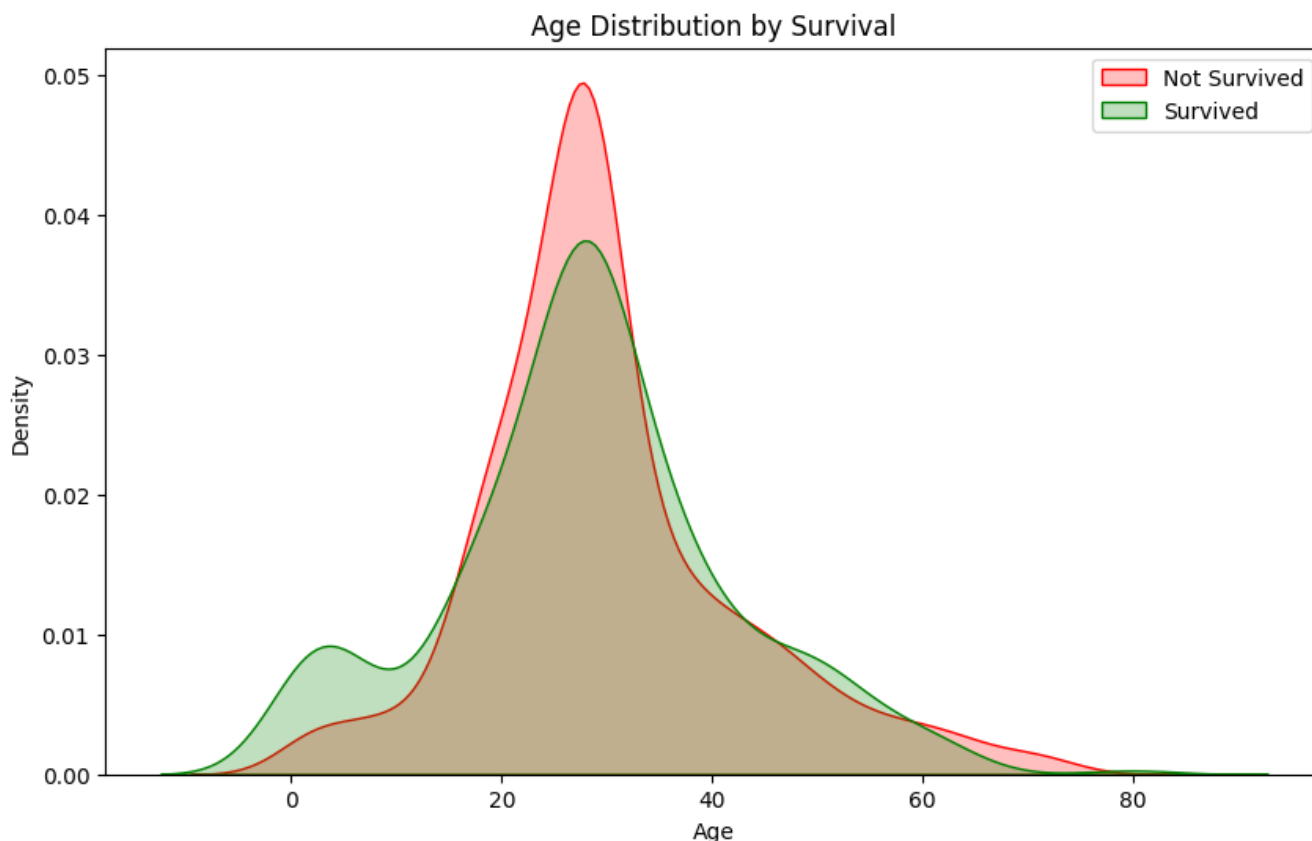


Figure 7: KDE curves of Age split by survival status

The overlapping KDE curves show that young children (under ~10) had a notably higher survival density, reflecting the "women and children first" evacuation priority.

6.4 Age Bins (AgeBin)

Continuous Age is discretised into five ordered categories: Child (0–12), Teen (12–18), Young (18–35), Adult (35–60), Senior (60–100). This helps models learn non-linear age effects more easily.

7. Data Preprocessing

7.1 Dropping Unnecessary Columns

Six columns are dropped before modelling: PassengerId (sequential ID with no signal), Ticket (inconsistent format), Cabin (77% missing), Name (Title already extracted), SibSp and Parch (replaced by FamilySize).

7.2 Outlier Detection — Box Plots

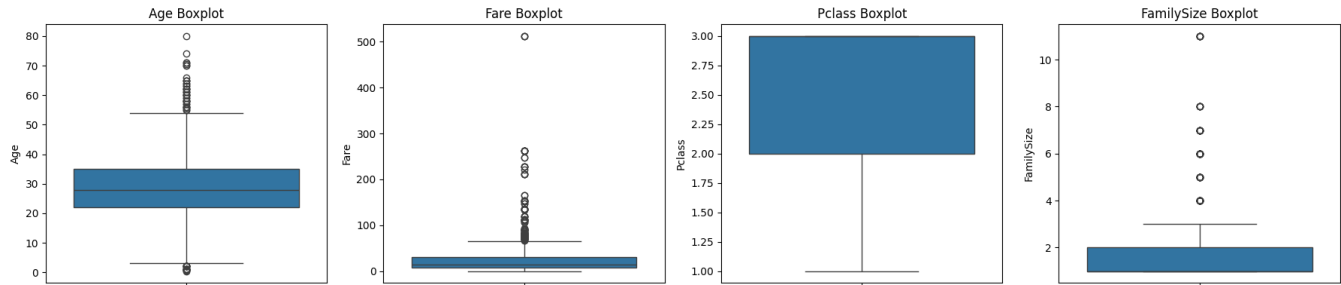


Figure 8: Box plots of Age, Fare, Pclass, and FamilySize

Box plots reveal the interquartile range (IQR), median, and outliers (points beyond $1.5 \times \text{IQR}$). Fare shows a long upper tail with expensive first-class fares.

7.3 Log Transformation of Fare

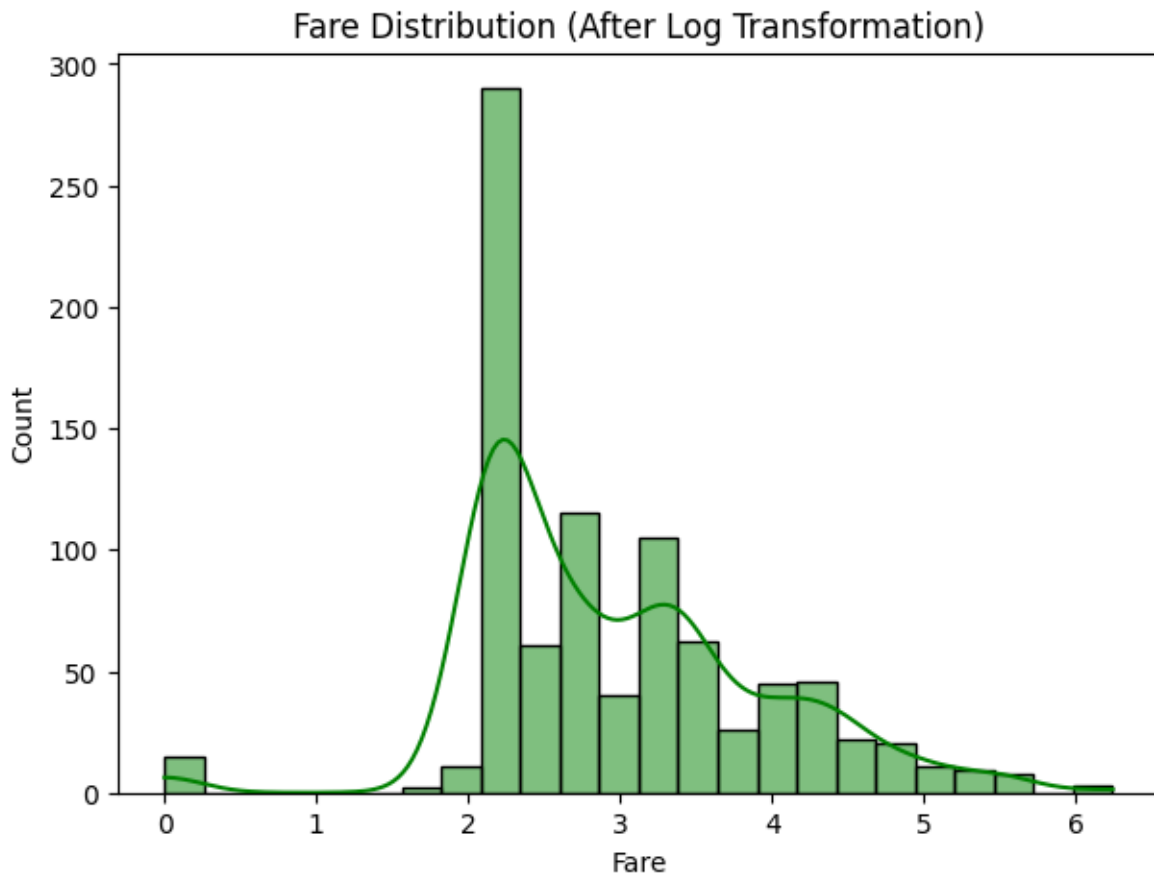


Figure 9: Fare distribution after $\log_{1p}$ transformation

Applying `np.log1p(Fare)` compresses the skewed distribution towards a more normal bell curve, which benefits linear and distance-based models.

7.4 Encoding Categorical Variables

Three encoding strategies are used:

- (1) Binary label encoding for Sex (male=0, female=1).
- (2) One-hot encoding via `pd.get_dummies()` for Embarked and Title, with `drop_first=True` to avoid the dummy variable trap.
- (3) Ordinal encoding for AgeBin, preserving the natural age order (Child=0 through Senior=4).

7.5 Feature Scaling

StandardScaler normalises numeric features to mean=0, std=1. It is fit only on training data and applied (transform-only) to test data to prevent data leakage. Scaled features: Age, Fare, Pclass, FamilySize, AgeBin.

7.6 Train / Validation Split

The labelled data is split 80/20 into training (712 rows) and validation (179 rows) sets using `stratify=y` to maintain the class ratio in both splits. `random_state=42` ensures reproducibility.

8. Model Training & Evaluation

Five classifiers are trained and evaluated. For each model we report training accuracy, validation accuracy, and the full classification report (precision, recall, F1).

8.1 Logistic Regression

Logistic Regression finds optimal feature weights by minimizing log-loss via iterative optimization (`max_iter=1000`). The sigmoid function maps the linear combination to a probability; threshold 0.5 determines the predicted class.

Metric	Train	Validation
Accuracy	0.8202	0.8324

Class	Precision	Recall	F1	Support
0	0.84	0.89	0.87	110
1	0.81	0.74	0.77	69
accuracy			0.83	179
macro avg	0.83	0.82	0.82	179
weighted avg	0.83	0.83	0.83	179

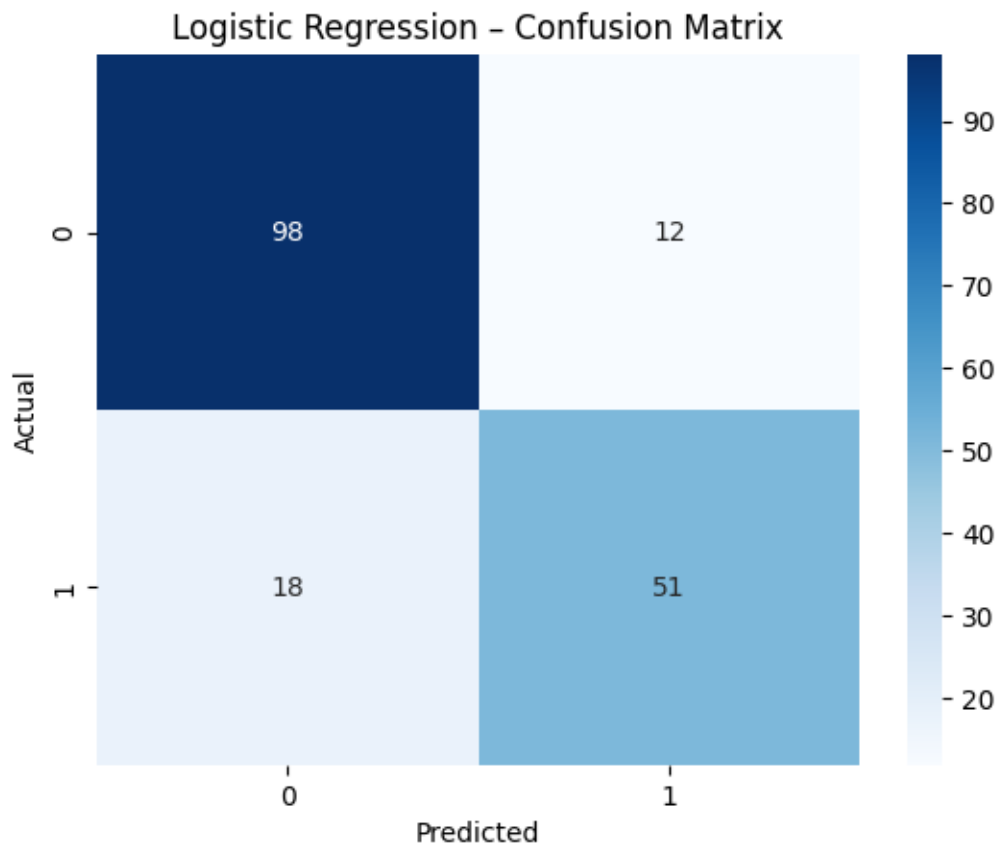


Figure 10: LR Confusion Matrix

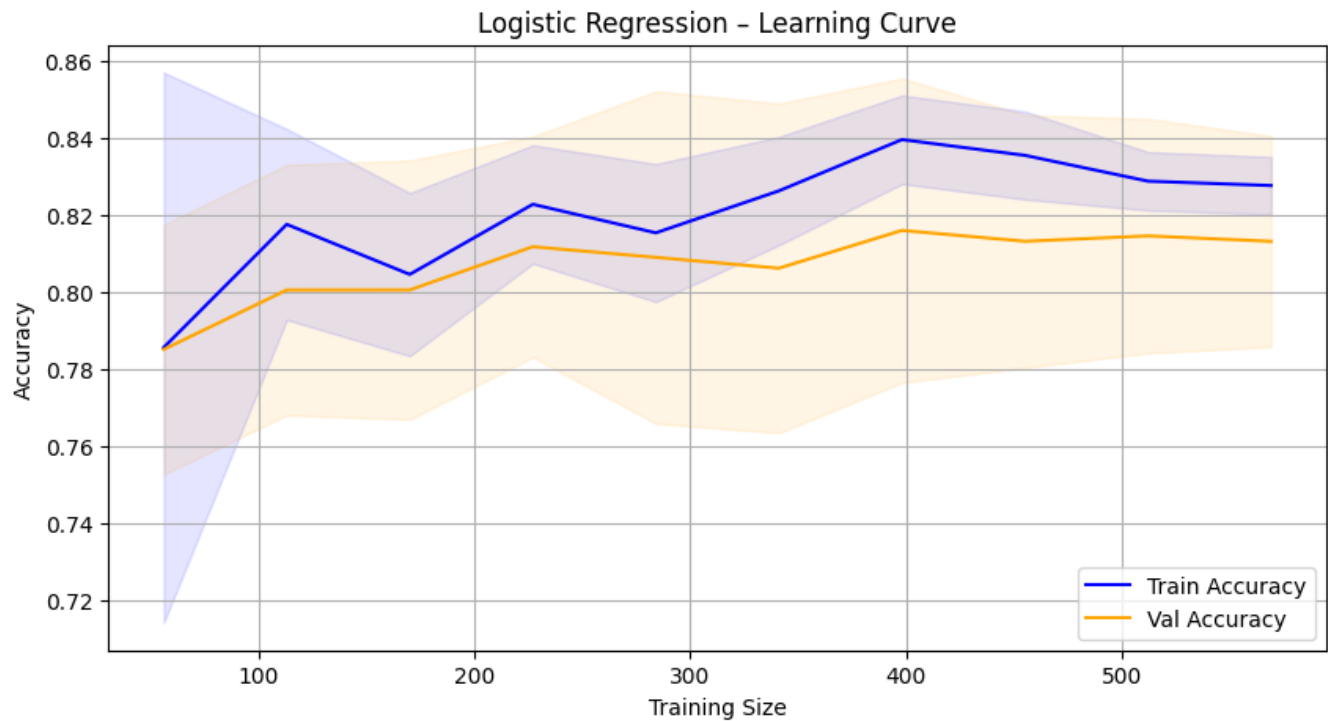


Figure 11: LR Learning Curve

8.2 Support Vector Machine (SVM)

SVC with default RBF kernel finds the maximum-margin hyperplane separating the two classes. The kernel trick maps data into higher dimensions where a linear boundary is feasible.

Metric	Train	Validation
Accuracy	0.8371	0.8268

Class	Precision	Recall	F1	Support
0	0.84	0.88	0.86	110
1	0.80	0.74	0.77	69
accuracy			0.83	179
macro avg	0.82	0.81	0.81	179
weighted avg	0.83	0.83	0.83	179

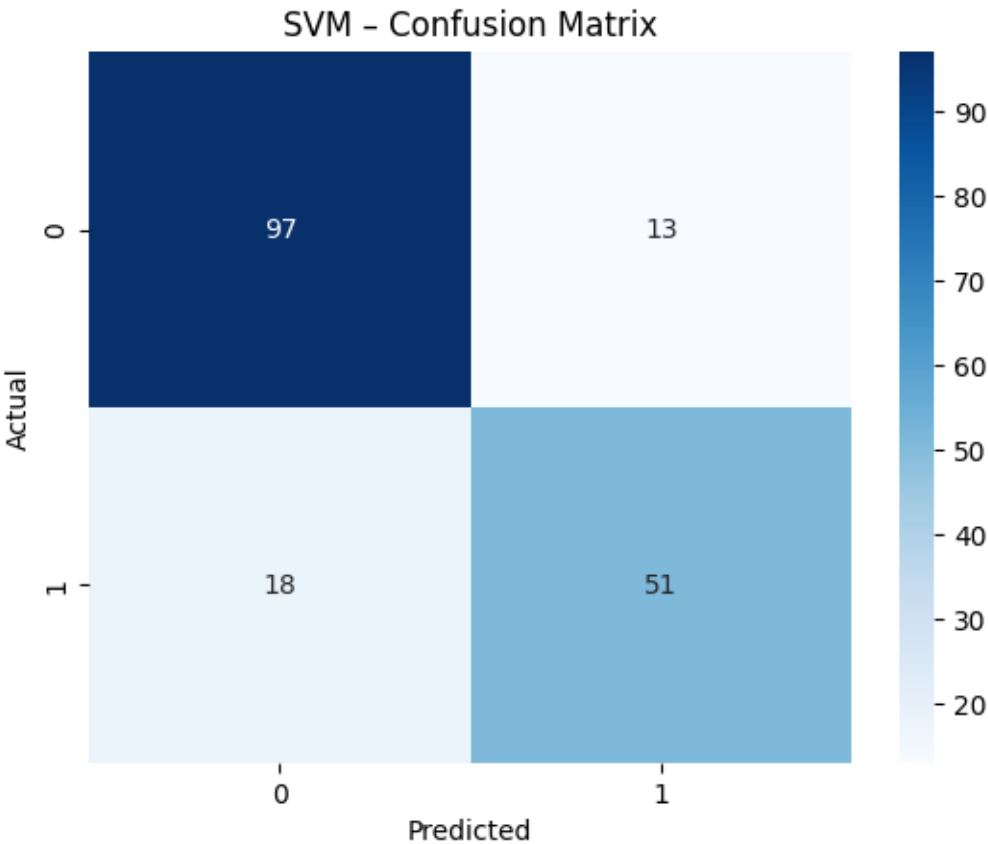


Figure 12: SVM Confusion Matrix

8.3 Decision Tree (max_depth=5)

The Decision Tree recursively splits data on the feature that best reduces impurity (measured by Gini). max_depth=5 prevents overfitting by limiting the number of sequential yes/no questions the tree can ask.

Metric	Train	Validation
Accuracy	0.8694	0.8324

Class	Precision	Recall	F1	Support
0	0.86	0.87	0.86	110
1	0.79	0.77	0.78	69
accuracy			0.83	179
macro avg	0.82	0.82	0.82	179
weighted avg	0.83	0.83	0.83	179

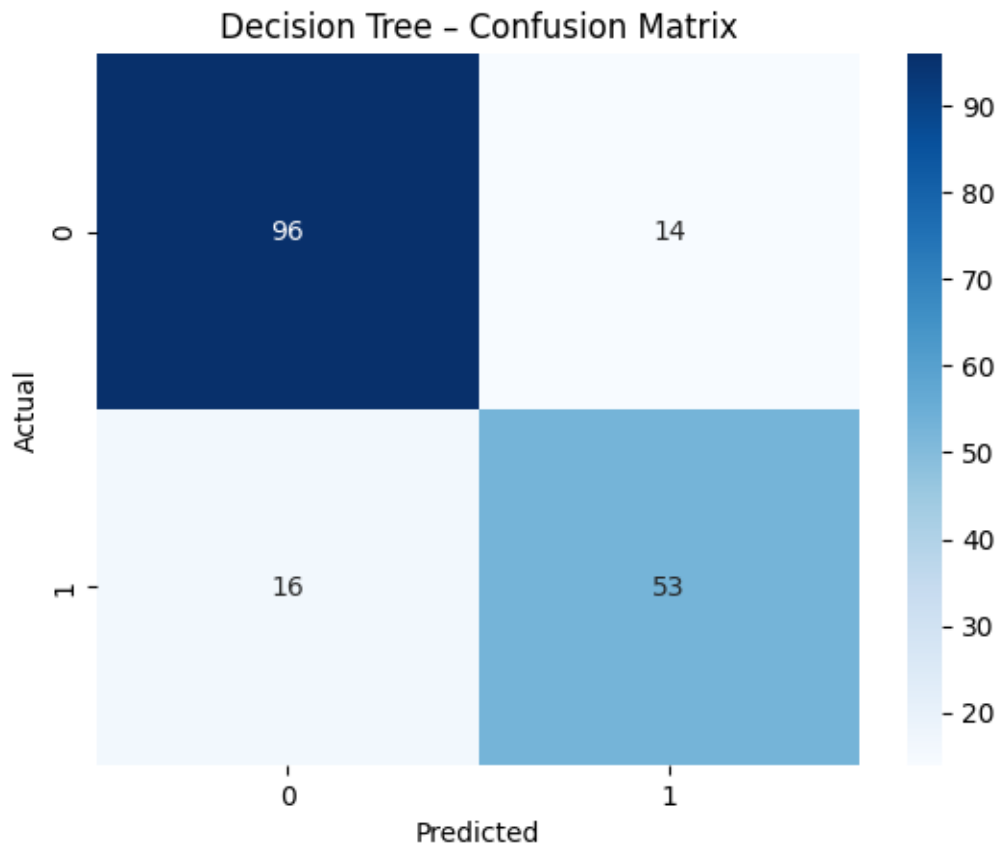


Figure 13: Decision Tree Confusion Matrix

8.4 Random Forest (max_depth=5)

Random Forest builds 100 decision trees, each on a bootstrap sample with a random feature subset at each split. The final prediction is a majority vote. This bagging approach drastically reduces variance compared to a single tree.

Metric	Train	Validation
Accuracy	0.8469	0.8212

Class	Precision	Recall	F1	Support
0	0.83	0.89	0.86	110
1	0.80	0.71	0.75	69
accuracy			0.82	179
macro avg	0.82	0.80	0.81	179
weighted avg	0.82	0.83	0.82	179

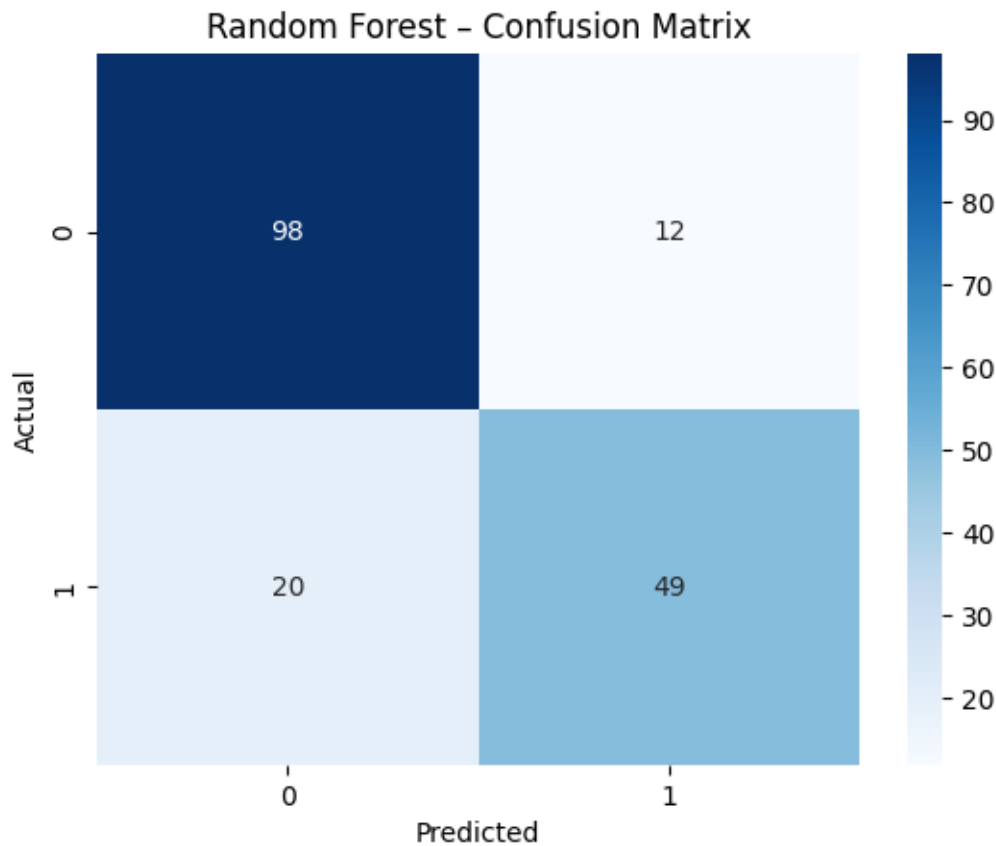


Figure 14: Random Forest Confusion Matrix

8.5 K-Nearest Neighbors (best K=12)

KNN classifies each passenger by finding the 12 most similar training passengers by Euclidean distance and taking a majority vote of their labels. K values from 1 to 20 are tested; the plot below shows validation accuracy vs K.



Figure 15: KNN validation accuracy for K=1 to 20 (best K=12)

Metric	Train	Validation
Accuracy	0.8399	0.8324

Class	Precision	Recall	F1	Support
0	0.82	0.94	0.87	110
1	0.87	0.67	0.75	69
accuracy			0.83	179
macro avg	0.84	0.80	0.81	179
weighted avg	0.84	0.83	0.83	179

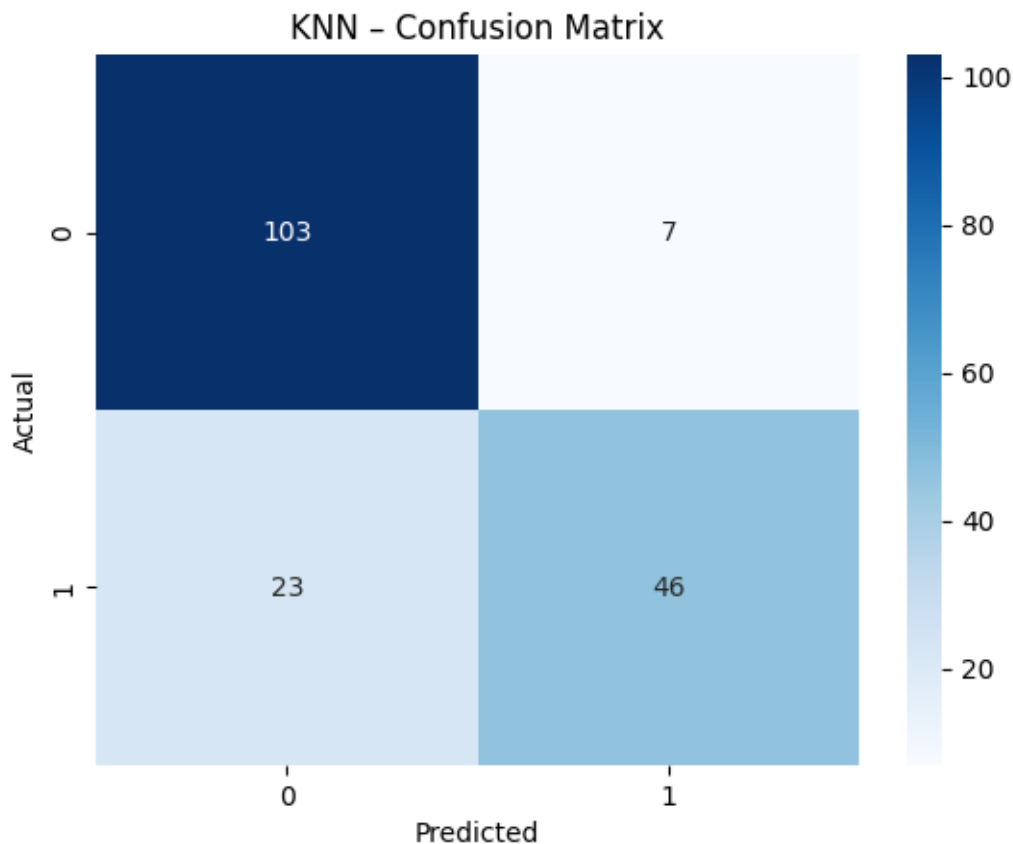


Figure 16: KNN Confusion Matrix (K=12)

9. Hyperparameter Tuning (GridSearchCV)

GridSearchCV exhaustively tests all combinations of hyperparameters using 5-fold stratified cross-validation on the training set, selecting the combination with the highest mean CV accuracy. `n_jobs=-1` uses all CPU cores in parallel.

9.1 Logistic Regression Tuning

Search space: `C` ∈ [0.01, 0.1, 1, 10, 100], `penalty` ∈ [l1, l2], `solver` = liblinear. Best parameters found: `C=0.1`, `penalty=l1`, `solver=liblinear`. Validation accuracy: 0.8492.

Metric	Train	Validation
Accuracy	0.8230	0.8492

Class	Precision	Recall	F1	Support
0	0.85	0.91	0.88	110
1	0.84	0.75	0.79	69
accuracy			0.85	179
macro avg	0.85	0.83	0.84	179
weighted avg	0.85	0.85	0.85	179

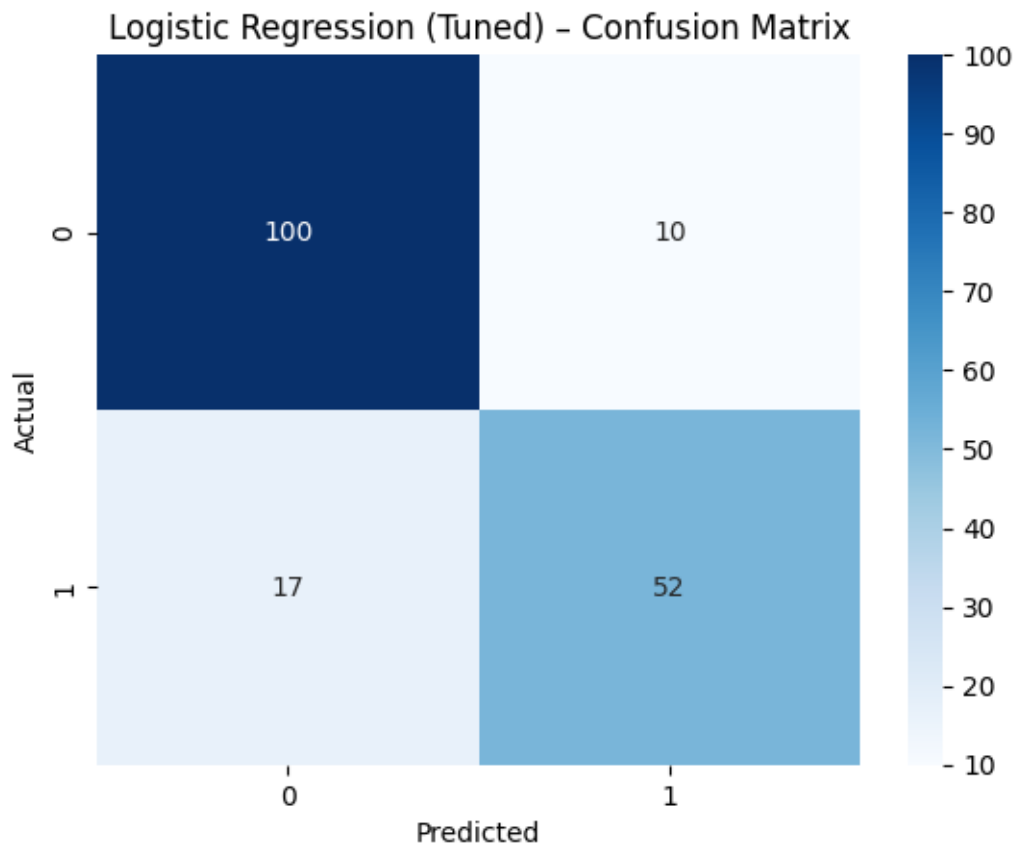


Figure 17: Logistic Regression (Tuned) Confusion Matrix

9.2 SVM Tuning

Search space: $C \in [0.1, 1, 10]$, $\text{kernel} \in [\text{rbf}, \text{linear}]$, $\text{gamma} \in [\text{scale}, \text{auto}]$. Best parameters: $C=0.1$, $\text{kernel}=\text{linear}$, $\text{gamma}=\text{scale}$. Validation accuracy: 0.8268.

Metric	Train	Validation
Accuracy	0.8511	0.8268

Class	Precision	Recall	F1	Support
0	0.83	0.91	0.87	110
1	0.83	0.70	0.76	69
accuracy			0.83	179
macro avg	0.83	0.80	0.81	179
weighted avg	0.83	0.83	0.82	179

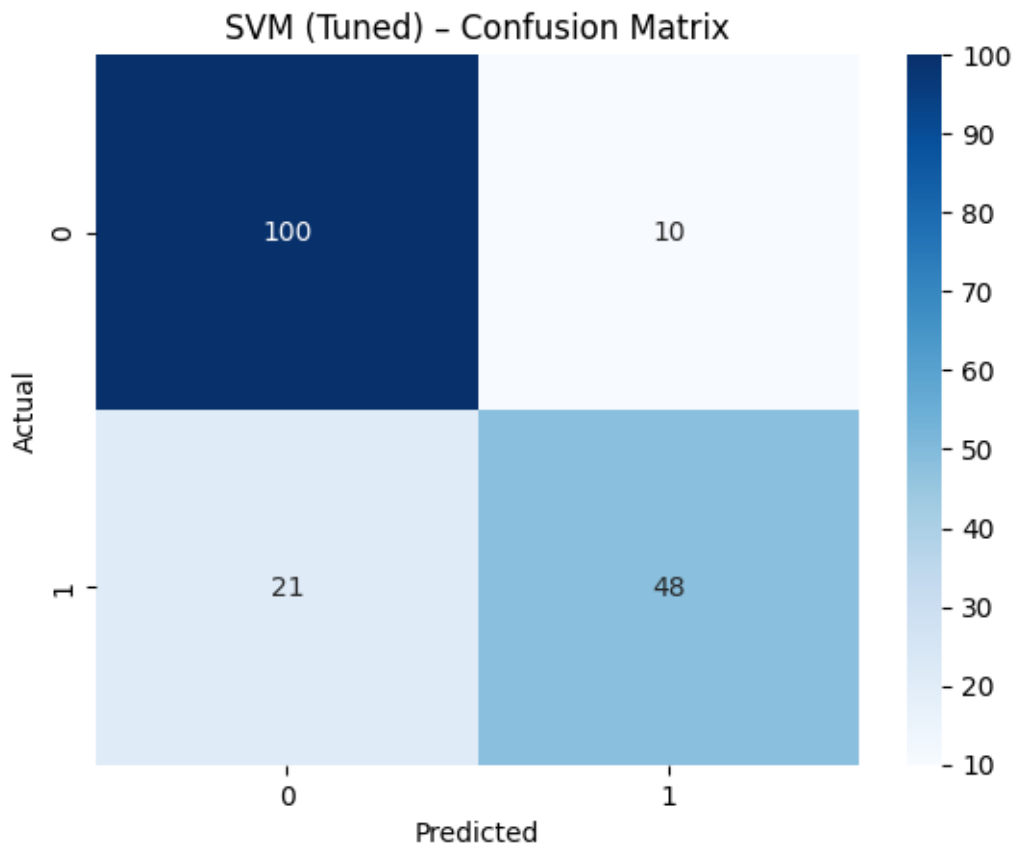


Figure 18: SVM (Tuned) Confusion Matrix

9.3 Decision Tree Tuning

Search space: max_depth = range(2,16), Best parameters: max_depth=7. Validation accuracy: 0.8436

Metric	Train	Validation
Accuracy	0.8919	0.8436

Class	Precision	Recall	F1	Support
0	0.86	0.89	0.88	110
1	0.82	0.77	0.79	69
accuracy			0.84	179
macro avg	0.84	0.83	0.83	179
weighted avg	0.84	0.84	0.84	179

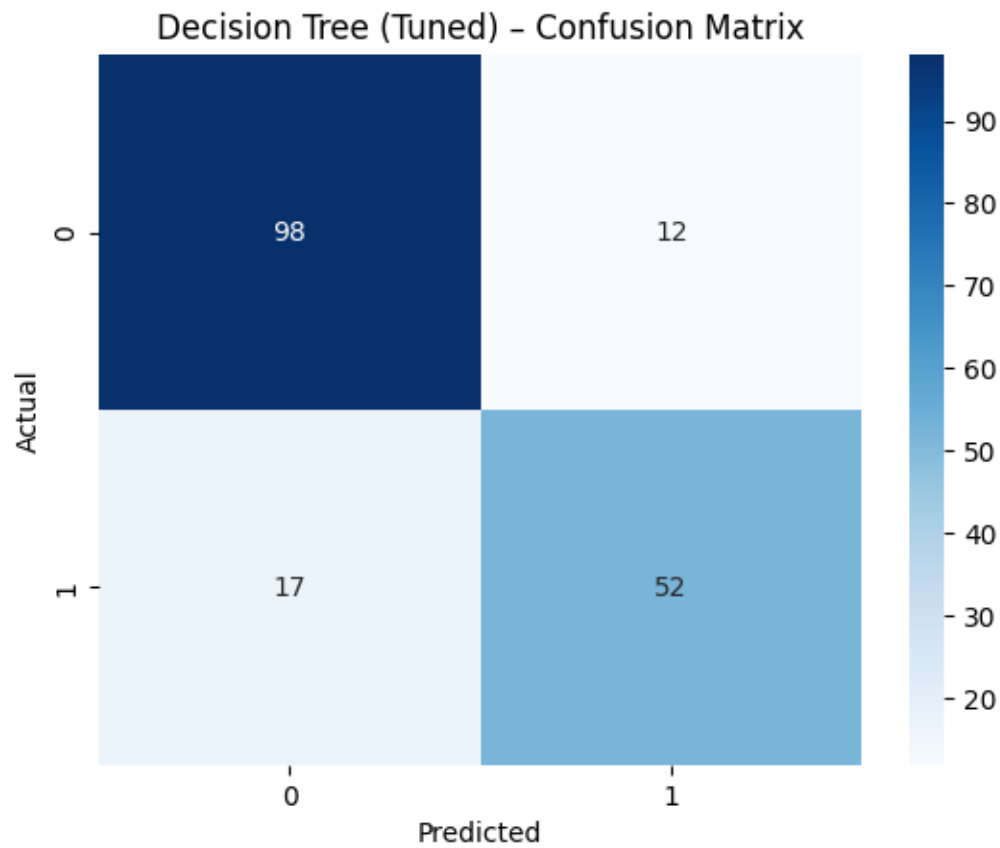


Figure 19: Decision Tree (Tuned) Confusion Matrix

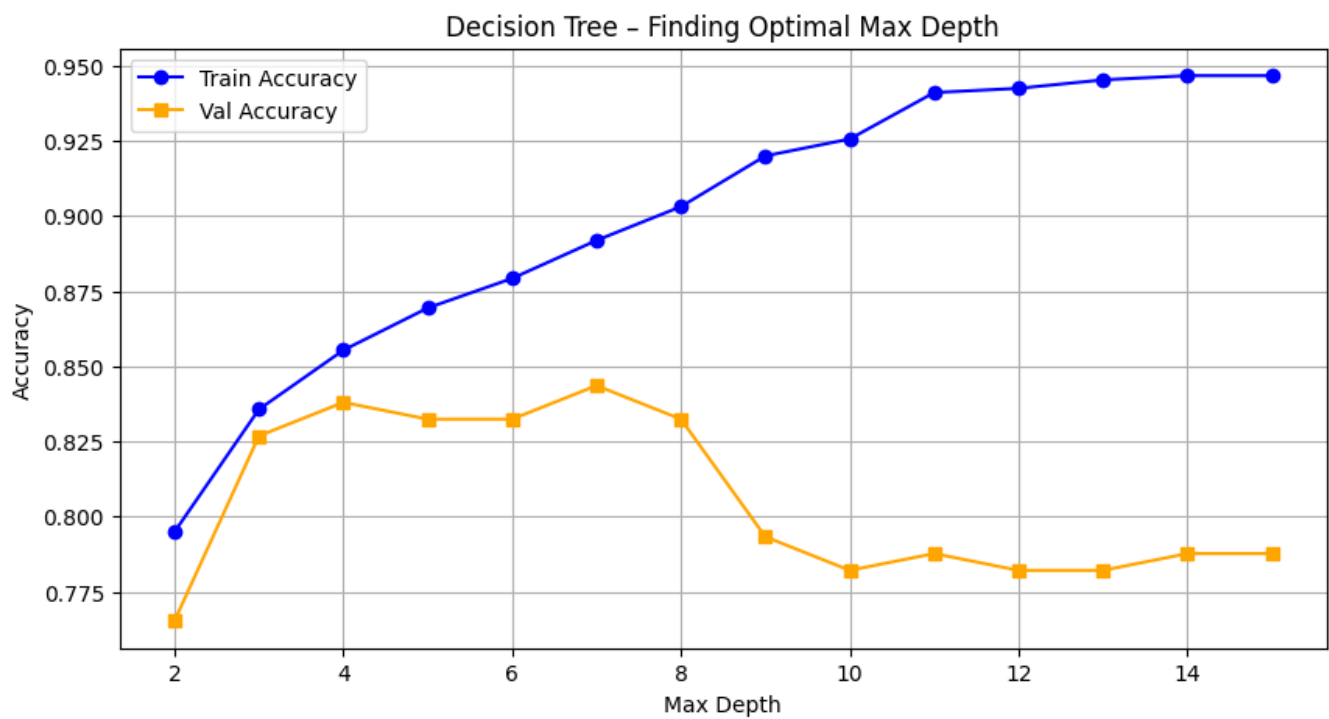


Figure 20: Decision Tree Max Depth

10. Model Comparison

All eight model variants (5 base + 3 tuned) are compared side-by-side. The table is sorted by validation accuracy descending.

Model	Train Acc	Val Acc
Logistic Regression (Tuned)	0.8230	0.8491
Decision Tree (Tuned)	0.8918	0.8435
Decision Tree	0.8693	0.8324
Logistic Regression	0.8202	0.8324
KNN (K=12)	0.8398	0.8324
SVM	0.8370	0.8268
SVM(Tuned)	0.8511	0.8268
Random Forest	0.8469	0.8212

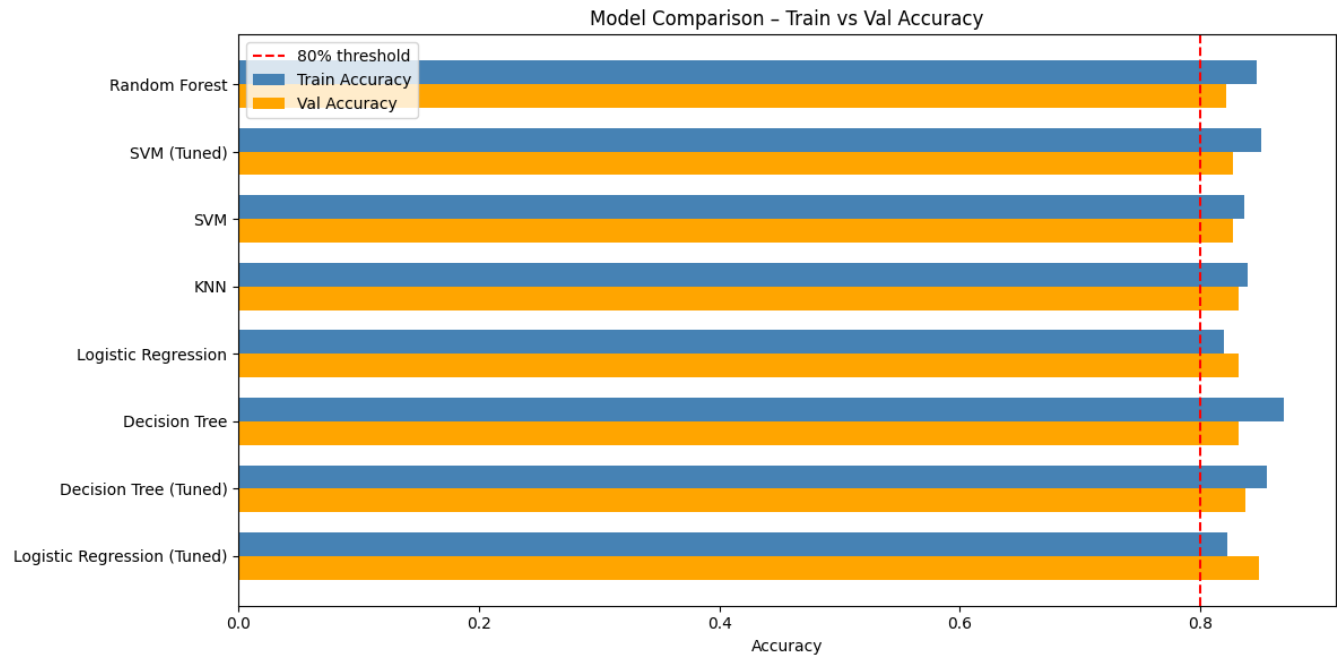


Figure 20: Train vs Validation accuracy — all models

11. Best Model Selection & Predictions

The best-performing model is automatically selected by comparing validation accuracy across all trained variants. The winner is LR (Tuned) with a validation accuracy of 0.8436 (84.36%).

This model is then used to generate predictions for all 418 test set passengers. Predictions are saved as a CSV file with both numeric (0/1) and readable (Died/Survived) labels.

12. Complete Pipeline Summary

Step	Description
Step 1: Load Data	pd.read_csv() loads training (891 rows) and test (418 rows) CSV files.
Step 2: Clean Data	Median imputation for Age/Fare, mode imputation for Embarked, Cabin dropped.
Step 3: EDA	Countplots, histograms, KDE plots, and correlation heatmap explore the data.
Step 4: Feature Engineering	Title extracted from Name, FamilySize and IsAlone created, AgeBin discretises Age.
Step 5: Drop Columns	PassengerId, Ticket, Cabin, Name, SibSp, Parch removed.
Step 6: Outlier Check	Box plots visualize outliers; log1p applied to Fare.
Step 7: Encode	Sex label-encoded, Embarked/Title one-hot encoded, AgeBin ordinal encoded.
Step 8: Scale	StandardScaler normalizes numeric features (fit on train, transform on test).
Step 9: Split	80/20 stratified train/validation split with random_state=42.
Step 10: Train	5 classifiers trained: Logistic Regression, SVM, Decision Tree, Random Forest, KNN.
Step 11: Tune	GridSearchCV with 5-fold CV tunes LR, SVM, and Decision Tree.
Step 12: Compare	All 8 model variants compared by train/val accuracy.
Step 13: Predict	Best model generates predictions on test set; saved to CSV.

"I hope this project meets your expectations."
